

Path-Gradient Estimators for Continuous Normalizing Flows

Lorenz Vaitl¹, Kim Nicoli^{1 2}, Shinichi Nakajima^{1 2 3}, Pan Kessel^{1 2}

Affiliations: ¹ Machine Learning Group, EECS, TU Berlin, Germany • ² BIFOLD, TU Berlin, Germany • ³ RIKEN Center for AIP, Tokyo, Japan

Proceedings of the 39th International Conference on Machine Learning (PMLR 162), Baltimore, USA, 2022

Correspondence: pan.kessel@tu-berlin.de

Overview

Variational Inference • CNFs • Path-gradients

Abstract & Motivation

- Path-gradient estimators can reduce gradient variance vs. total-derivative estimators.
- Prior work focused on simple variational families (e.g., Gaussians); limited expressivity.
- We propose a path-gradient estimator for continuous normalizing flows (CNFs).
- Algorithm: efficient, same memory as standard training; slightly higher per-iter runtime.
- Empirically superior on VAEs and lattice field theory tasks.

Key idea: path-wise gradients for CNFs

Lower variance → Faster convergence

Introduction

- VI applied to high-dimensional domains: VAEs, graphics, physics, chemistry.
- Expressive variational densities via normalizing flows enable complex modeling.
- CNFs (NODE-based) allow free-form Jacobians, symmetry inductive biases.
- Roeder et al. (2017): path-gradient shows lower variance, faster convergence.
- Challenge: Efficient path-gradient for (continuous) normalizing flows.

Gap

Efficient path-gradients for CNFs

Related Work

- Path-gradients: Roeder et al. (2017); extensions to IWAE/RWS/JVAE, α -divergences.
- Normalizing flows with path-wise estimators: Agrawal et al. (2020) — higher memory/runtime.
- Our contribution: efficient path-gradient for CNFs with constant memory and lower runtime than prior NF approach.

Contribution vs prior

Half memory vs Agrawal et al.

Variational Inference & Path-Gradients

- Reparameterized variational family: $x = g_{\theta}(z), z \sim q_z$.
- Optimize reverse KL: $KL(q_{\theta} || p)$ with reparameterization trick.
- Total derivative splits into path term and score term; score has Fisher information variance.
- At optimum $q_{\theta^*}=p$: path-gradient vanishes; score variance remains $\rightarrow$ higher variance for total gradient.
-  Expectation: path-gradient lower variance near convergence $\Rightarrow$ better training stability.

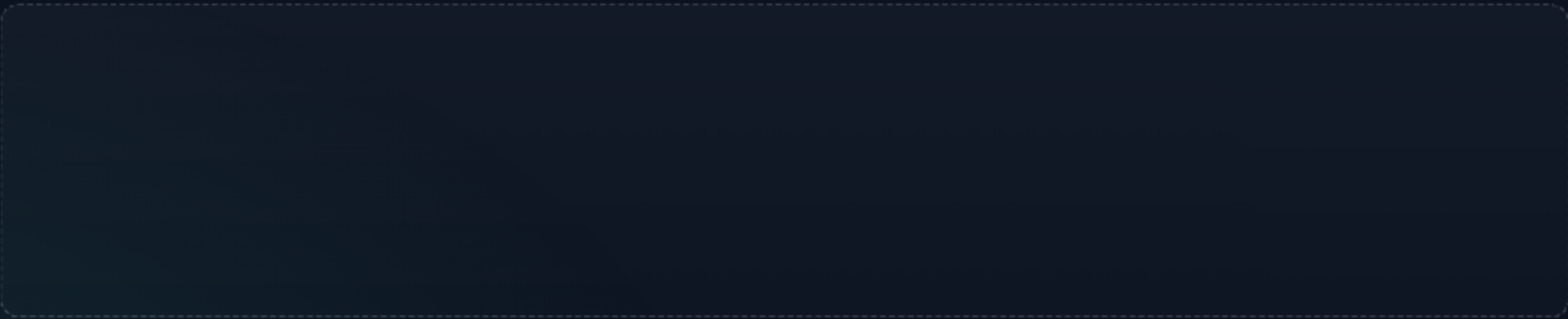
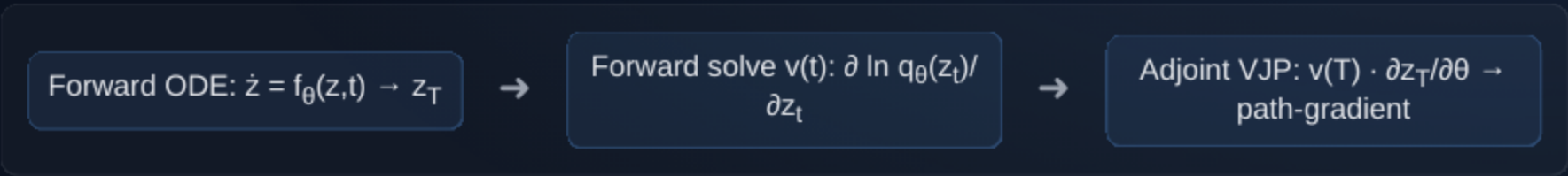


Figure 1: Path-gradient norm (lower curve) stays far below total-gradient norm during ϕ^4 training (L=12), indicating lower-variance, stabler updates.

Path-Gradients for Simple Densities

- Two-step trick (Roeder et al.): sample with θ , evaluate log-density with θ' , then $\partial/\partial\theta$ at θ .
- Works when sampling decouples from log-density evaluation (e.g., Gaussians).
- Problem for NFs: log-density involves $|\partial g_{\theta}/\partial z|$ from sampling path.
- Agrawal et al. (2020) use model cloning (θ and θ'): doubles memory and increases runtime.

Path-Gradient Workflow (Lead Visual)



Lead schematic: forward state, forward $v(t)$ via Theorem 4.1, then reverse-mode VJP yield the path-gradient with constant memory.

- Compute $\partial \ln q_{\theta}(z_T)/\partial z_T$ by solving a forward ODE; then apply adjoint for the vector–Jacobian product.

Continuous Normalizing Flows (CNFs)

- Flow via ODE: $\dot{x} = f_{\theta}(x,t)$, map $g_{\theta}: z_0 \rightarrow z_T$.
- Log-density evolution: uses trace of Jacobian (Hutchinson estimator in practice).
- Adjoint method: compute gradients with constant memory via terminal value problem.
- Total derivative cost ≈ 3 forward passes (forward + adjoint with recomputation).

Our Path-Gradient for CNFs: Overview

- Goal: compute $\partial/\partial\theta \ln q_\theta(z_T(\theta))$ via path term only.
- Key step: obtain $\partial \ln q_\theta(z_T)/\partial z_T$ by solving a forward ODE (Theorem 4.1).
- Then compute vector–Jacobian product with reverse-mode (adjoint) to get path-gradient.
-  Cost $\approx$ 4 forward passes; constant memory; shared computations reduce overhead.



Figure 2: Our estimator needs ~ 4 forward-pass equivalents vs ~ 3 for total gradient and ~ 6 with $2\times$ memory for path-wise NF (Agrawal et al., 2020). Note: $+1$ pass stems from the forward ODE for $\partial \ln q/\partial z$.

Efficiency Constant memory • $\sim 4/3$ runtime

Theorem 4.1 (Operational Form)

- Solve an initial value problem to obtain $v(t) = \partial \ln q_\theta(z_t)/\partial z_t$ forward in time.
- Initialize $v(0)$ from base density; integrate alongside state ODE.
- Compute $v(T)$ and use adjoint VJP to accumulate path-gradient w.r.t. θ .

Pseudo-steps: 1) Forward ODE: $z_0 \rightarrow z_T$ and track $v(t)$ via Theorem 4.1 2) Compute $v(T) = \partial \ln q(z_T)/\partial z_T$ 3) Reverse (adjoint) for VJP: $v(T) \cdot \partial z_T/\partial \theta \rightarrow$ path-gradient

Implementation Forward $v(t)$ + adjoint VJP

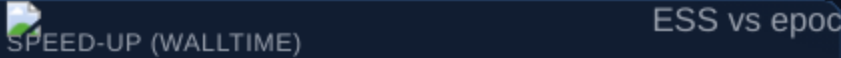
Practical Considerations

- Forward solve for $v(t)$ requires no intermediate state recomputation.
- Synergies: shared $\partial f/\partial z$ terms reduce effective overhead below 33%.
- Works with fixed and adaptive ODE solvers; same memory as total gradient.

Robustness Scalable and memory efficient

Experiments: Lattice Field Theory (ϕ^4)

- Symmetry-aware CNF (de Haan et al., 2021); $L \times L$ lattices with periodic BCs.
- Train $KL(q_\theta||p)$ with total vs path-gradient estimators.
- Evaluate via ESS using self-normalized importance sampling.

 SPEED-UP (WALLTIME) ESS vs epochs for $L=12, 20, 32$

Faster convergence

Consistently higher with path

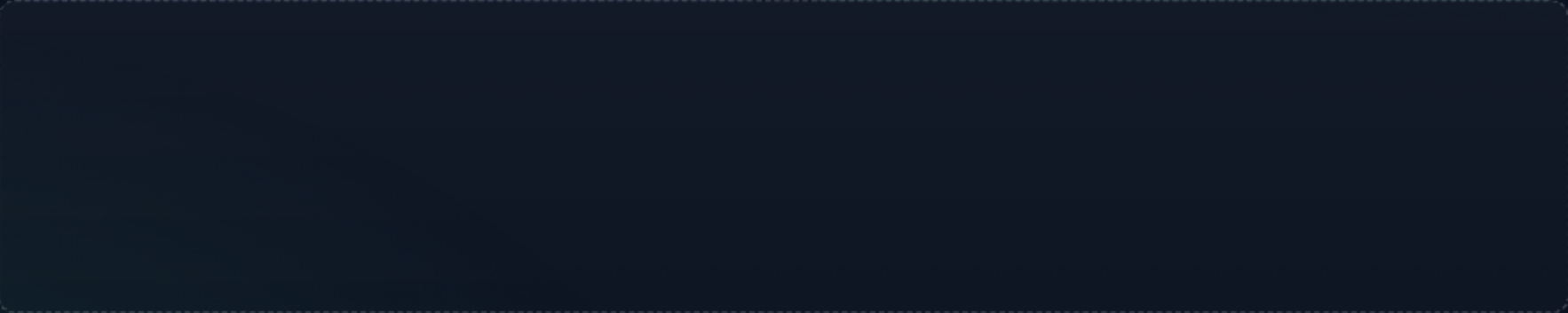


Figure 3: ESS over training for $L=12, 20, 32$ (higher is better). Path $>$ total across all L .

- Axes: x = training progress; y = ESS (%). Comparison: path-gradient reaches higher ESS sooner than total.

Experiments: VAEs with FFJORD (Grathwohl et al., 2019)

- Datasets: MNIST • Omniglot • Caltech Silhouettes • Frey Faces
- Training: early stopping; adaptive-step ODE solver
- Metric: Negative ELBO (lower is better)

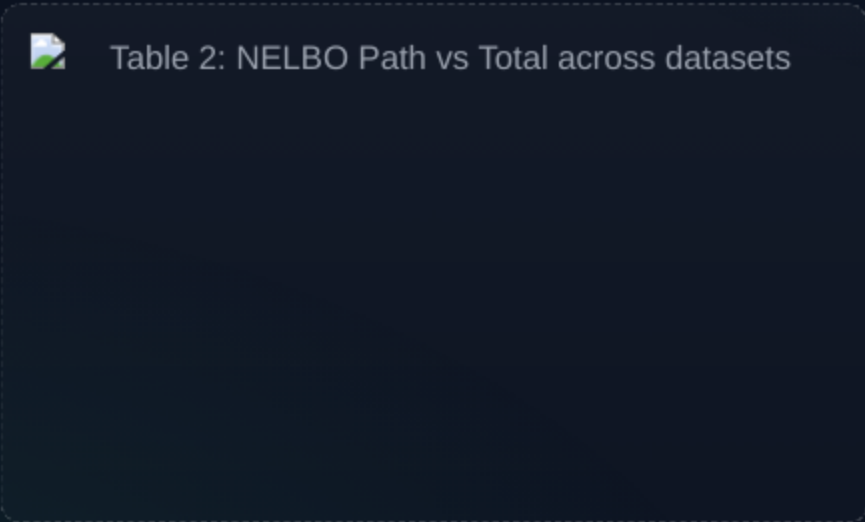


Table 2: Path-gradient achieves lower (better) NELBO than total on MNIST, Omniglot, Caltech Silhouettes, and Frey Faces using FFJORD (Grathwohl et al., 2019).

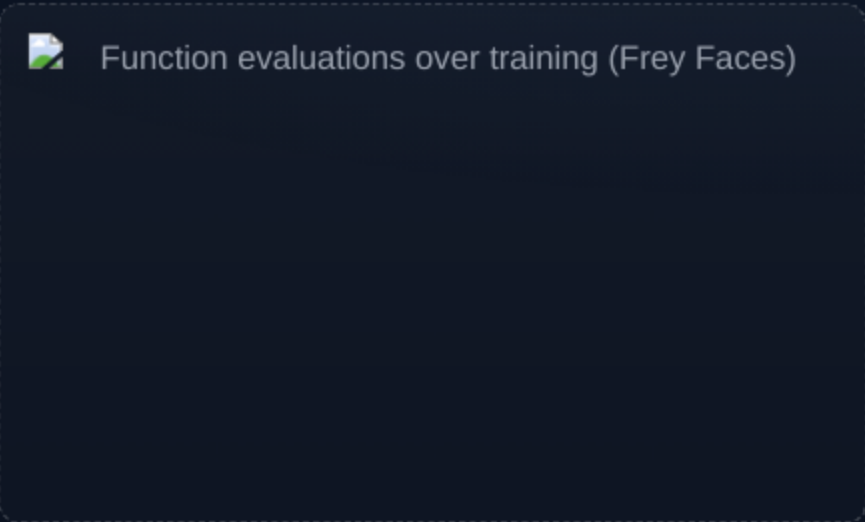


Figure 5: Number of ODE function evaluations (adaptive solver) on Frey Faces shows no noticeable difference between path and total; both track similarly.

Results Summary

- Plug-and-play replacement of total gradient with path-gradient improves performance.
- Higher ESS (LFT) and better NELBO (VAEs) under comparable or fixed runtime.
- Constant memory; larger feasible batch sizes vs prior NF path-gradient method.

VARIANCE

Lower near optimum

RUNTIME

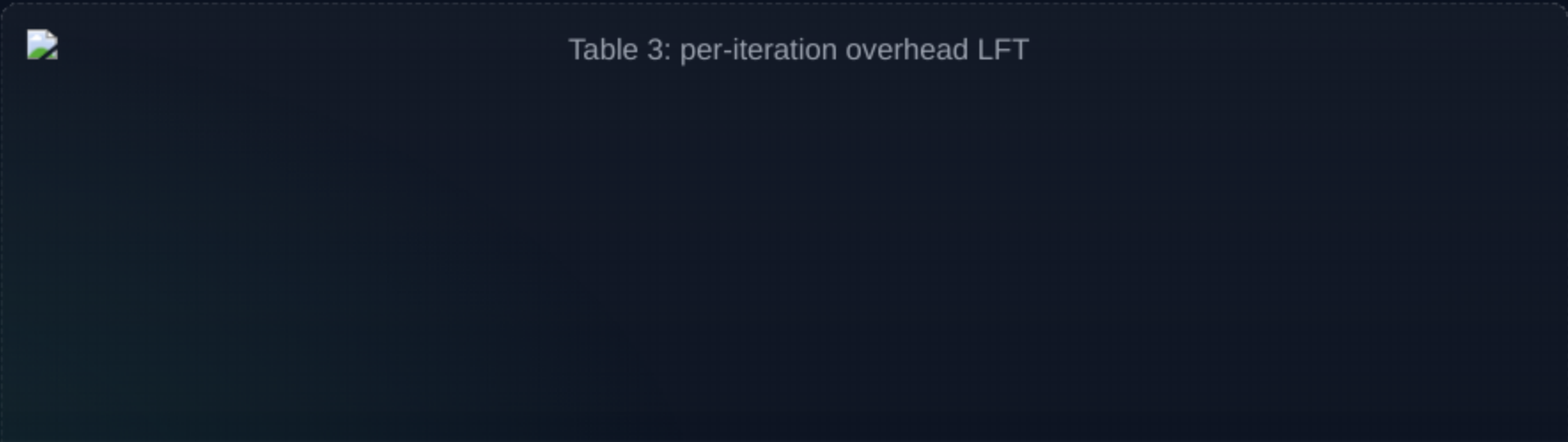
~4/3 of total; yet faster to good solutions

MEMORY

Constant; no model cloning

Runtime Comparison and Baselines

- CNF/LFT: path-gradient per-iter overhead +14% (12×12), +12% (20×20), +5% (32×32) on a single A100 GPU; below ~33% estimate.
- Path estimator far faster than path-wise NF with model cloning (Agrawal et al., 2020) and uses half the memory.
- VAE with FFJORD (Grathwohl et al., 2019): overall runtime comparable; no notable NFE change.



Main Weakness and Evidence

- Overhead per iteration exists (path ≈ 4 fp vs total ≈ 3 fp); can be noticeable on small CNFs.
- Example (LFT, Table 3): +14% per-iter time on 12×12 lattice; benefits grow with problem size as overhead drops to +5% at 32×32.
- Instability not observed in our reported experiments; theory currently justifies variance gains mainly near convergence (Section 3).

Limitations & Discussion

- Theory explains variance reduction near convergence; full-phase theory remains open.
- Empirically beneficial across entire training in tested domains.
- Future: broaden theory; explore further solver/architecture synergies.

Open questions

[Toward broader theory](#)

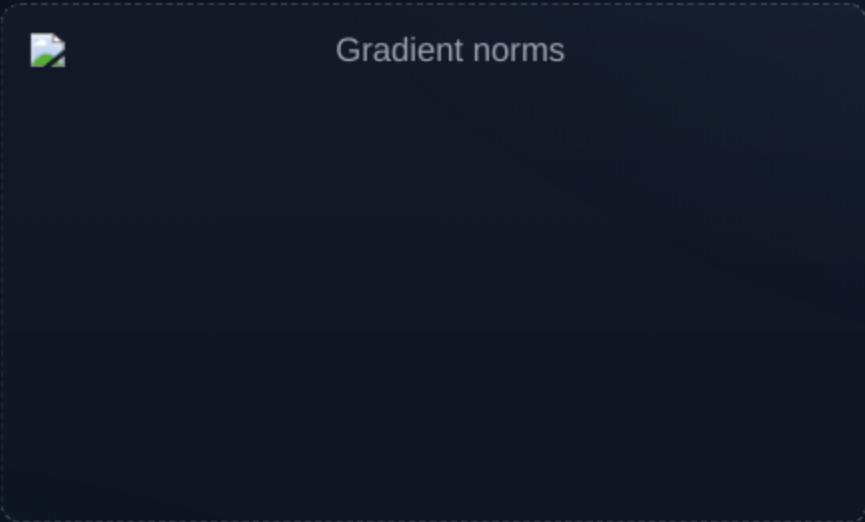
References (Selected)

- Roeder et al., 2017 — Sticking the Landing: Simple, Lower-Variance Gradient Estimators for VI.
- Chen et al., 2018 — Neural Ordinary Differential Equations; CNFs.
- Grathwohl et al., 2019 — FFJORD: Free-form Jacobian of Reversible Dynamics.
- Agrawal et al., 2020 — Path-wise gradients for NFs (ablation; higher cost).
- de Haan et al., 2021 — Symmetry-preserving CNFs for lattice field theory.
- Kingma & Welling, 2014; Rezende et al., 2014 — VAEs and reparameterization trick.

Code

github.com/lenz3000/ffjord-path

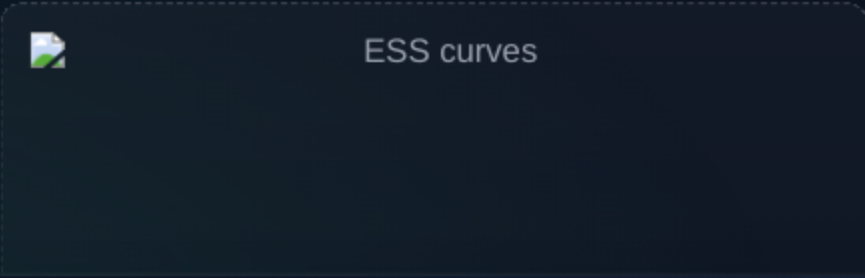
Figures & Tables



Panel A (Fig. 1): Path-gradient norms remain much smaller than total-gradient norms over epochs (ϕ^4 , $L=12$) $\rightarrow$ reduced variance near convergence.



Panel B (Fig. 2): Runtime/memory comparison — ours $\approx$ 4 forward passes, total $\approx$ 3, path-wise NF (Agrawal et al., 2020) $\approx$ 6 with 2 $\times$ memory.



Conclusion

- New path-gradient estimator for CNFs: efficient, constant memory, easy to integrate.
- Empirical gains across VAEs and lattice field theory; faster convergence, better final metrics.
- Practical recommendation: prefer path-gradients for CNF-based VI tasks.

Contact: pan.kessel@tu-berlin.de • Code: github.com/lenz3000/ffjord-path

Thank you

[Questions?](#)